



DEPARTMENT OF
COMPUTER SCIENCE



UNDERSTANDING PROBLEMS AND PROBLEM TYPES

DR JEREMIAH O. BANDELE

PhD Electrical/Electronic Engineering
University of Nottingham



LEARNING OBJECTIVES

At the end of this lesson, you should be able to:

- Identify problems and types of problems
- Describe routine and non-routine problems
- Describe solvable and unsolvable problems



WHAT ARE PROBLEMS?

- ❖ People face various straightforward and complex problems everyday.
- ❖ Problems can be ill-defined without clear goals, expected solutions, and solution paths.
- ❖ Problems can also be well defined with clear goals, expected solutions or solution paths.



ROUTINE PROBLEMS

Any problem that can be solved with an arithmetic operation is known as a **routine problem**.



ROUTINE PROBLEMS

- Provides a question that needs an answer, in addition to some required facts or numbers.
- Can be solved with algorithms.
- Is solely about identifying the most appropriate algorithm to apply to the problem.
- Can be solved very quickly.



ROUTINE PROBLEMS: EXAMPLE

Some examples of routine problems are given below

- Calculate the area of a 150m x 2000m parking space?



ROUTINE PROBLEMS: EXAMPLE

- Dan has \$500, he gave some of the money to his friends. Now, he has \$287.
How much did he give his friend?



NON-ROUTINE PROBLEMS

Any problem that does not have a standard method for solving it is known as a **non-routine problem**.



NON-ROUTINE PROBLEMS

- Use procedures and answers that are not immediately clear.
- Requires a distinct and unconventional thinking process based on originality and creativity.
- Can usually be solved using various approaches
- Can take some time to solve.



NON-ROUTINE PROBLEMS

To solve non-routine problems, a 'novel' method has to be developed.

This method will involve:

- Studying the problem critically
- Gathering relevant information
- Identifying and implementing an effective strategy



NON-ROUTINE PROBLEMS

Some examples of non-routine problems are given below;

- ❖ Provide an approximate number of hairs on your head?



NON-ROUTINE PROBLEMS

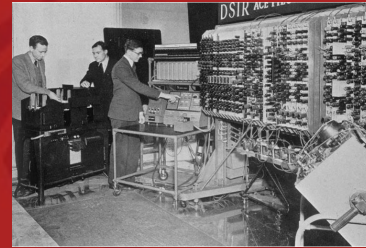
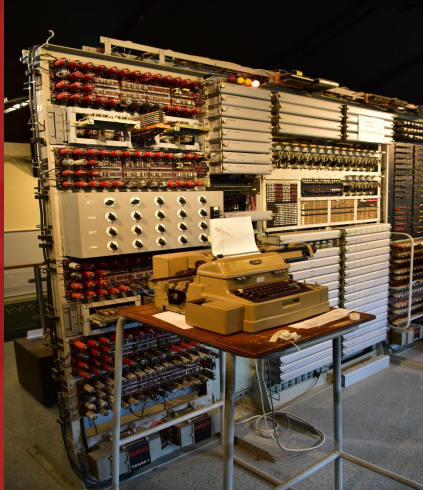
Some examples of non-routine problems are given below:

There are 50 questions in an exam. 2 marks are deducted for any wrong response and 4 marks are awarded for every correct response. Bridget scored 165 marks. How many correct responses did she provide?



THE UNIVERSAL TURING MACHINE

Alan Turing introduced a 'universal computing machine', now referred to as the 'universal Turing machine (UTM)'.





HOW THE UTM WORKS?

- The UTM has a 'pre-loaded' single, fixed table of instructions.
- Any task we can write an instruction table for, can be implemented by the UTM.



HOW THE UTM WORKS?

- Our instruction table (or program) only needs to be put on the tape of the UTM.
- These instructions can be in the form of a string of letters.
- After the UTM reads the instructions, it implements the task on its tape.



MID LESSON QUESTIONS

2. Which of the following is not an attribute of routine problems?
- a) Provides a question that needs an answer, in addition to some required facts or numbers.
 - b) Can be solved with algorithms.
 - c) **Can usually be solved using various unconventional approaches.**
 - d) Is solely about identifying the most appropriate algorithm to apply to the problem.



THE UTM AND THE MODERN COMPUTER

The modern computer is based on the UTM.



THE UTM AND THE MODERN COMPUTER

- Turing's most significant contributions to the advent of modern computer are:
- The idea of manipulating the functionalities of a modern computer by storing encoded instructions in the computer's memory.
- His demonstration that a single machine of fixed structure can carry out every computation that the UTM can carry out.



SOLVABLE AND UNSOLVABLE PROBLEMS

- Any mathematical problem that can be solved with a systematic method (i.e. an algorithm) can be solved by the UTM.
- They are known as **solvable problems**.



SOLVABLE AND UNSOLVABLE PROBLEMS

- Mathematical problems that cannot be solved by the UTM (or with an algorithm) are known as **unsolvable problems**.
- An example of a problem that are not solvable with any systematic method is the problem where puzzles are sorted into those that will 'come out' and those that will not.



SOLVABLE PROBLEMS

Problems that are solvable can be based on data structures.

- Examples of such data structures are vectors, stacks, queues, arrays, heaps, trees, graphs, and linked lists.



SOLVABLE PROBLEMS

Problems that are solvable can also be based on algorithms.

- Examples of such problems are sorting problems, divide and conquer problems, recursive problems, searching problems, and text processing problems.



SUBSTITUTION PUZZLES

- ❖ Puzzle such as a chess problem (or any other puzzle) can be re-expressed as a substitution puzzle.
- ❖ A simple example of the substitution puzzle is to find out the possibility of obtaining 'BOOOB' from the word 'BOB' by replacing 'OB' with 'BOOB' and by replacing 'BOB' with 'O'.
- ❖ Let us try to see if this is possible.



SUBSTITUTION PUZZLES: SOLUTION TO EXAMPLE

- Obtaining 'BOOOB' from the word 'BOB' by replacing 'OB' with 'BOOB' and by replacing 'BOB' with 'O'.



THE MEANING OF UNSOLVABLE

- Unlike what we might think, 'unsolvable problems' are not problems for which no solution can possibly be found.
- Deciding whether or not substitution puzzles come out is an unsolvable problem.



THE MEANING OF UNSOLVABLE

- Another way to put this is that there is no systematic method for deciding whether or not substitution puzzles come out.
- So in the real sense, we actually have ‘unsolvable decision problems’, and not just ‘unsolvable problems’.



DECIDABLE AND UNDECIDABLE PROBLEMS

- A decidable problem is one for which a systematic way can be found to solve a problem accurately.
- If a problem exist such that no algorithm can give an accurate true/false for every input value, that problem is undecidable (a subsets of unsolvable problems).



DECIDABLE AND UNDECIDABLE PROBLEMS

Examples of undecidable problems are;

- Does my code have a bug?
- Halting problem.



HALTING PROBLEM

- The problem is to determine whether a running program will eventually halt or run infinitely.
- Let us see how it works.



FURTHER READING RESOURCES

- Killen, R. (2007). Teaching strategies for outcomes-based education. Juta and Company Ltd.
- Baumanns, L. (2022). Mathematical problem posing: Conceptual considerations and empirical investigations for understanding the process of problem posing. Springer Nature.
- Dawson, Jr, J. W. (2007). The Essential Turing: Seminal Writings in Computing, Logic, Philosophy, Artificial Intelligence, and Artificial Life plus The Secrets of Enigma, by Alan M. Turing (author) and B. Jack Copeland (editor).



REFERENCES

1. Monica, A. (2017). *Brainstorming: Thinking-Problem Solving Strategy*. Er. Manoj Kumar. *Int. Journal of Engineering Research and Application*, 7(3), 33-37.
2. Spielman, R. M., Dumper, K., Jenkins, W., Lacombe, A., Lovett, M., & Perlmutter, M. (2021). *Problem Solving*. Psychology-H5P Edition.



DEPARTMENT OF
COMPUTER SCIENCE



THANK
YOU